PRÁCTICA OPENCODE UT2. ANALIZAR UNA ARQUITECTURA CON UN AGENTE

Antes de empezar

opencode instalado (ver la **guía de instalación**) y el proyecto en un repositorio Git, que es tu botón de deshacer. Comprueba con `/models` qué modelo tienes activo: con los modelos gratuitos tendrás que ser más explícito en los prompts y cometerán más errores, lo que viene bien para lo que se pide aquí. Necesitas además la aplicación `tienda-libros` de la carpeta `recursos`, copiada a tu carpeta de trabajo, y un visor de Markdown que dibuje diagramas Mermaid (Visual Studio Code con la extensión «Markdown Preview Mermaid Support», o el propio GitHub). No hace falta levantar la aplicación.

- Trabaja en **modo Plan** (tecla Tab) hasta que el plan te convenza; solo entonces pasa a **Build**.
- Revisa cada cambio antes de aceptarlo (`git diff`). Si no sabes explicar una línea, no la aceptes.
- **Verifica ejecutando**: la evidencia es la salida del comando, no la respuesta del agente.
- Anota cada prompt en `prompts.md`: qué pediste, qué hizo y qué corregiste.
- Nada de contraseñas ni datos personales en el chat: los modelos gratuitos son de prueba y no admiten información confidencial.
- Si el modelo se atasca, da vueltas o va muy lento, divide la petición en pasos o cambia de modelo con `/models`.

Ejercicios

Todo el trabajo se recoge en un documento `docs/arquitectura.md` dentro del proyecto, que escribes **tú** con la ayuda del agente.

1) **Agente en solo lectura.** Crea `opencode.json` con `"edit": "deny"`, `"bash": "ask"` y `"share": "disabled"`, arranca opencode en **modo Plan** y pide un resumen de la aplicación. Después pídele expresamente que modifique `docker-compose.yml`: captura el rechazo. Explica por qué en una fase de análisis conviene un agente que no pueda escribir. (1p)
2) **Inventario de componentes.** Pide al agente la lista de servicios y rellena esta tabla comprobando **cada celda** en los ficheros: servicio, imagen o construcción, puerto publicado al exterior, puerto interno, volúmenes, de qué servicios depende y qué hace. Indica cuántos servicios son accesibles desde fuera del equipo y por qué. (1,5p)
3) **Recursos de la aplicación.** Clasifica todos los recursos del proyecto (contenido estático, código de servidor, datos y esquema, dependencias, configuración, secretos, registros) indicando en qué fichero o servicio vive cada uno y cuáles **no deben subirse al repositorio**. Comprueba si el proyecto tiene `.gitignore` y propón uno. (1,5p)

4) **Recorrido de una petición.** Pide al agente dos diagramas **Mermaid**: el de `GET /api/libros` (incluyendo la caché) y el de `POST /api/pedidos` (incluyendo la cola de facturas y el worker). Comprueba que el diagrama coincide con el código de `api/src/server.js` y `worker/worker.js`, corrígelo y verifica que se dibuja en el visor. (1,5p)
5) **Modelo de arquitectura.** Pregunta al agente qué modelo de arquitectura sigue la aplicación y por qué. Contrasta su respuesta con la teoría de la unidad y con los ficheros: justifica el modelo con **evidencias concretas** (número de despliegues independientes, quién habla con la base de datos, cómo se comunican los servicios) y señala qué parte de la respuesta del agente es una generalidad de manual que no se sostiene aquí. Añade ventajas e inconvenientes de este diseño para una tienda pequeña. (1,5p)
6) **Ficha de requerimientos de implantación.** Con ayuda del agente, redacta la ficha para poner esta aplicación en producción: hardware o instancia, software y versiones, red (dominio, DNS, puertos, cortafuegos), seguridad y secretos, entornos, datos y copias de seguridad, monitorización y registros, fases del despliegue y plan de vuelta atrás. La ficha debe distinguir con claridad **lo que la aplicación ya trae** de **lo que hay que añadir** para producción. (2p)
7) **Caza de afirmaciones falsas.** A lo largo de la práctica, el agente dirá cosas que no son ciertas para esta aplicación. Recoge al menos **tres**, cada una con la frase del agente, el fichero y la línea que la desmienten y la afirmación correcta. (1p)

Reto (ampliación)

Repite el ejercicio 2 con dos prompts distintos: uno vago («explícame la arquitectura») y otro preciso, citando ficheros con `@docker-compose.yml` `@api/src/server.js` y diciendo qué formato quieres. Compara las dos respuestas y explica qué cambia. Publica después `docs/arquitectura.md` en el repositorio de la unidad para que se vea el diagrama renderizado.

Registro de prompts

Mantén durante toda la práctica el fichero `prompts.md` con una tabla como esta (una fila por prompt importante):

Nº	Prompt (resumido)	Qué hizo el agente	Qué corregí yo
1	Explica qué hace este proyecto y qué ficheros lo componen	Resumen correcto; se inventó una carpeta tests/	Le indiqué que no existe y lo comprobé con ls

Entrega

Entrega por el aula virtual un PDF con las capturas y las respuestas de cada ejercicio (debajo de cada uno), el fichero `prompts.md` y el enlace al repositorio. Una captura de la respuesta del agente **sin la comprobación** correspondiente no puntúa.